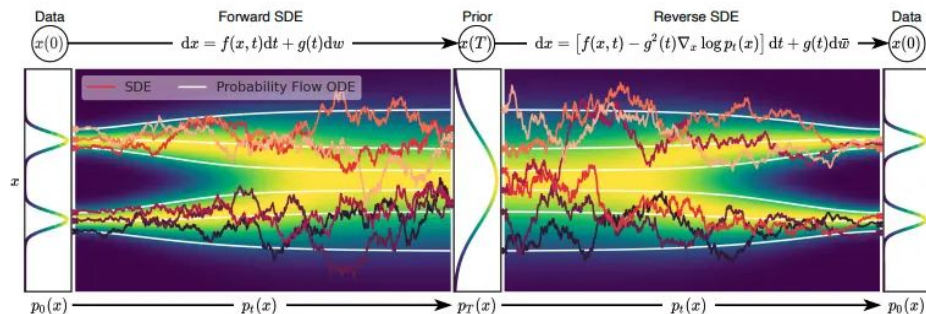


An Introduction to Flow Matching

Anthony Wang (University of California, San Diego)

Limitations of Diffusion

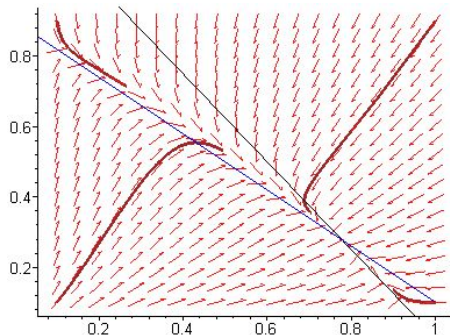
- Computationally expensive!
 - Diffusion models must learn to carefully denoise a vector/matrix at each time step
 - Neural networks often have large and complex architectures
 - Mathematics behind diffusion are also difficult to grasp (but still very “beautiful”!)
- Inherently stochastic/non-deterministic
 - Brownian motion and stochastic differential equations form the foundation
 - Results in chaotic, “jagged” trajectories as the distribution moves from Gaussian noise to the target



Ordinary Differential Equations

- In its simplest form, an ordinary differential equation (ODE) can be expressed as follows:

$$\begin{aligned}\frac{d}{dt}X_t &= u_t(X_t) \\ X_0 &= x_0\end{aligned}$$

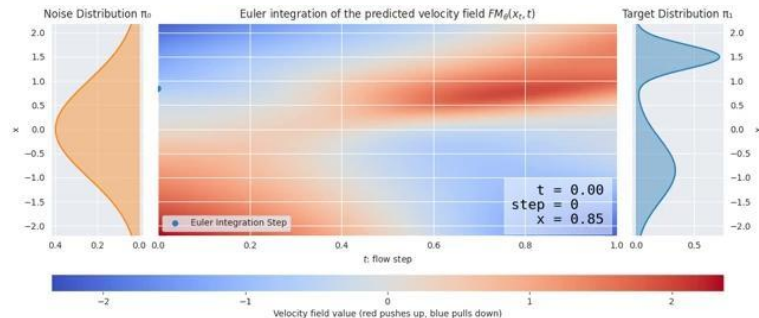


- Here, u is a vector field which can be mathematically defined as:

$$u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, (x, t) \rightarrow u_t(x)$$

Flow Matching Algorithm

- The essence of flow matching is to apply the idea of an ODE to random variables:
 - Recall that in diffusion we start with random noise and denoise so it becomes a sample from a target distribution (image, video, etc.)
 - What if we learned to “transport” the random noise directly to the desired distribution by using a vector field?
 - Vector field gives “velocity” that the random variable should move in to reach target distribution



Flow Matching Algorithm

- We would like to learn a neural network u_θ that takes in a random variable X_t , a time t , and outputs the predicted velocity:
 - A natural loss function to minimize is the mean squared error (MSE) loss:

$$\mathcal{L}_{FM} = \mathbb{E} [\|u_\theta(X_t, t) - v(X_t, t)\|_2^2]$$

- During training, need a way to get samples of trajectory at time t . A popular and straightforward choice is linear interpolation between noise and data:

$$X_t = (1 - t)X_0 + tX_1, t \in [0, 1]$$

- Under this choice of trajectory, the sample velocity is therefore

$$v(X_t, t) = X_1 - X_0$$

- The final loss function is now

$$\mathcal{L}_{FM} = \mathbb{E} [\|u_\theta(X_t, t) - (X_1 - X_0)\|_2^2]$$

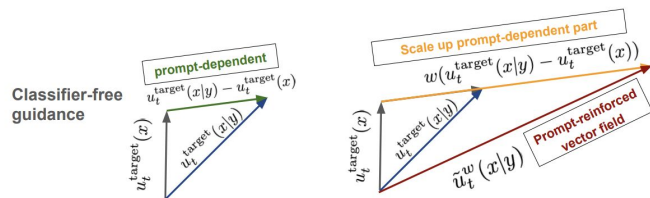
Adding Guidance to Flow Matching

- To add conditioning, we pass in context embedding c (i.e a text prompt) to the model setup to get a “conditional vector field”: $u_{\theta}(X_t, t, c)$
- Recall for classifier-free guidance in the context of diffusion, we utilize the unconditional score to avoid training an extra classifier

- We use an analogous approach here with the “unconditional vector field”:

$$\tilde{u}_{\theta}(X_t, t, c) = u_{\theta}(X_t, t, \emptyset) + \sigma \cdot (u_{\theta}(X_t, t, c) - u_{\theta}(X_t, t, \emptyset))$$

- The unconditional vector field is learned by randomly dropping context embeddings during training and replacing it with a null token



Final Training Loop

- We can now state the entire training loop

Algorithm 5 Classifier-free guidance training for Gaussian probability path $p_t(x|z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$

Require: Paired dataset $(z, y) \sim p_{\text{data}}$, neural network u_t^θ

- 1: **for** each mini-batch of data **do**
- 2: Sample a data example (z, y) from the dataset.
- 3: Sample a random time $t \sim \text{Unif}_{[0,1]}$.
- 4: Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
- 5: Set $x = \alpha_t z + \beta_t \epsilon$
- 6: With probability p drop label: $y \leftarrow \emptyset$
- 7: Compute loss

$$\mathcal{L}(\theta) = \|u_t^\theta(x|y) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon)\|^2$$

- 8: Update the model parameters θ via gradient descent on $\mathcal{L}(\theta)$.
 - 9: **end for**
-

Inference/Generation Process

- With the neural network trained, we now have a vector field that we can use for inference.
 - How to sample from a vector field given an initial condition? Euler's method!

Algorithm 1 Sampling from a Flow Model with Euler method

Require: Neural network vector field u_t^θ , number of steps n

- 1: Set $t = 0$
 - 2: Set step size $h = \frac{1}{n}$
 - 3: Draw a sample $X_0 \sim p_{\text{init}}$
 - 4: **for** $i = 1, \dots, n$ **do**
 - 5: $X_{t+h} = X_t + hu_t^\theta(X_t)$
 - 6: Update $t \leftarrow t + h$
 - 7: **end for**
 - 8: **return** X_1
-

Applications of Flow Matching

- Stable Diffusion (text-to-image): From version 3 onwards, Stable Diffusion switched to flow matching over the traditional diffusion algorithms used in earlier models
- Meta MovieGen (text-to-video): Used flow matching with a large transformer-based neural network as the backbone

References

- https://diffusion.csail.mit.edu/2026/docs/lecture_notes.pdf
- https://peterroelants.github.io/posts/flow_matching_intro/
- <https://harshm121.medium.com/flow-matching-vs-diffusion-79578a16c510>